Empresa Ejemplo S.A.
Gerencia de Arquitectura y Desarrollo de Software

Estándar de Desarrollo Seguro

Aplicaciones web, móviles, de escritorio y APIs

Código	Versión	Fecha de aprobación	Estado
EST-SEG-001	2.1	2026-03-10	Vigente

Aprobado por: Comité de Arquitectura y Seguridad de la Información

Documento de prueba con contenido ficticio. La empresa y las reglas son un ejemplo para pruebas de un asistente RAG; no corresponden a ninguna organización real.

Control de cambios

Versión	Fecha	Descripción del cambio
1.0	2022-05-02	Versión inicial basada en OWASP Top 10 2017.
2.0	2024-09-16	Alineación con OWASP ASVS 4.0.3 y Top 10 2021. Nuevo capítulo de aplicaciones móviles (MASVS).
2.1	2026-03-10	Se agregan reglas de JWT, deserialización y escritorio. Referencias a OWASP API Security Top 10 2023.

Contenido

Control de cambios . . . 2
1. Objetivo . . . 3
2. Alcance . . . 3
3. Definiciones y niveles de cumplimiento . . . 3
4. Principios generales . . . 3
5. Autenticación y gestión de sesiones . . . 4
6. Control de acceso . . . 6
7. Validación de entradas y codificación de salidas . . . 6
8. Criptografía y gestión de secretos . . . 8
9. Registro de eventos, errores y monitoreo . . . 9
10. Configuración segura y dependencias . . . 10
11. Aplicaciones móviles . . . 10
12. Aplicaciones de escritorio . . . 12
13. Excepciones . . . 12
14. Verificación del cumplimiento . . . 13

1. Objetivo

Establecer los controles mínimos de seguridad que deben cumplir las aplicaciones desarrolladas o adquiridas por la empresa, de modo que las vulnerabilidades más frecuentes se prevengan durante el diseño y la construcción y no se descubran recién en producción.

2. Alcance

Aplica a todas las aplicaciones web (Angular + ASP.NET Core), APIs, aplicaciones móviles (.NET MAUI, Android e iOS nativas) y aplicaciones de escritorio (.NET WPF y WinForms), sean desarrolladas por equipos internos o por proveedores. Este estándar prevalece sobre cualquier otro estándar de desarrollo en materia de seguridad. Las referencias externas se detallan en EST-REF-001.

3. Definiciones y niveles de cumplimiento

Término	Definición
Obligatorio	La regla debe cumplirse en todo desarrollo nuevo y en todo cambio sobre código existente. Su incumplimiento bloquea el pase a producción salvo excepción aprobada.
Recomendado	Buena práctica esperada. Su incumplimiento no bloquea el pase, pero debe justificarse en la revisión de código.
Prohibido	Práctica que no debe aparecer en el código. Equivale a una regla obligatoria en sentido negativo.
Excepción	Autorización temporal (máximo 6 meses) para no cumplir una regla obligatoria. La aprueba el Comité de Arquitectura y, si la regla es de seguridad, también Seguridad de la Información. Se registra en el formulario FOR-ARQ-07.
ASVS	OWASP Application Security Verification Standard, versión 4.0.3 adoptada por la empresa.
MASVS	OWASP Mobile Application Security Verification Standard, versión 2.x.
Dato sensible	Credenciales, tokens, número de documento de identidad, datos de tarjeta, datos de salud, información financiera de clientes y cualquier dato clasificado como Confidencial.

4. Principios generales

4.1 SEG-GEN-001 · Seguridad desde el diseño

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V1.1.2; OWASP A04:2021

Todo sistema nuevo o cambio de arquitectura debe contar con un modelo de amenazas (metodología STRIDE) revisado por Seguridad de la Información antes de iniciar la construcción. El modelo se actualiza cuando se agregan integraciones, nuevos tipos de usuario o nuevos datos sensibles.

4.2 SEG-GEN-002 · Mínimo privilegio

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V4.1.3; OWASP A01:2021

Cada componente (cuenta de servicio, usuario de base de datos, identidad administrada, rol de aplicación) recibe solo los permisos estrictamente necesarios. Está prohibido usar cuentas con privilegios de administrador (por ejemplo `sa` o `db_owner` en SQL Server) para la operación normal de una aplicación.

4.3 SEG-GEN-003 · Denegar por defecto y fallar de forma segura

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V4.1.5; OWASP A01:2021

Si no existe una regla de autorización explícita, el acceso se deniega. Ante un error inesperado (excepción, servicio de autorización no disponible) la aplicación debe negar la operación, nunca permitirla.

Ejemplo incorrecto

```
try { autorizado = await _permisos.TienePermisoAsync(usuario, "PAGOS"); }
catch (Exception) { autorizado = true; }   // si falla el servicio, se deja pasar
```

Ejemplo correcto

```
try { autorizado = await _permisos.TienePermisoAsync(usuario, "PAGOS"); }
catch (Exception ex)
{
    _logger.LogError(ex, "Fallo al verificar permisos de {Usuario}", usuario.Id);
    autorizado = false;                    // ante la duda, se deniega
}
```

5. Autenticación y gestión de sesiones

5.1 SEG-AUT-001 · Autenticación centralizada

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V2.1; OWASP A07:2021

Las aplicaciones de colaboradores deben autenticarse contra Microsoft Entra ID mediante OpenID Connect. Las aplicaciones para clientes externos usan el proveedor de identidad corporativo de clientes. Está prohibido crear almacenes de usuarios y contraseñas propios en la base de datos de una aplicación, salvo excepción aprobada.

5.2 SEG-AUT-002 · Almacenamiento de contraseñas

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V2.1.1; ASVS V2.1.7; ASVS V2.4.1; OWASP A02:2021

Cuando exista una excepción que permita contraseñas locales: longitud mínima de 12 caracteres, verificación contra listas de contraseñas filtradas y almacenamiento con un algoritmo de derivación de claves con sal (PBKDF2 con al menos 600 000 iteraciones, bcrypt o Argon2id). En .NET se usa `PasswordHasher<T>` de ASP.NET Core Identity. Está prohibido almacenar contraseñas cifradas de forma reversible o con hash MD5, SHA-1 o SHA-256 sin sal.

Ejemplo incorrecto

```
using var md5 = MD5.Create();
var hash = Convert.ToHexString(md5.ComputeHash(Encoding.UTF8.GetBytes(password)));
usuario.Clave = hash;
```

Ejemplo correcto

```
var hasher = new PasswordHasher<Usuario>();
usuario.ClaveHash = hasher.HashPassword(usuario, password);
// Verificación:
var resultado = hasher.VerifyHashedPassword(usuario, usuario.ClaveHash, intento);
```

5.3 SEG-AUT-003 · Autenticación multifactor

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** OWASP A07:2021; MASVS-AUTH-3

Es obligatorio el segundo factor (MFA) para: accesos administrativos, accesos remotos, operaciones monetarias iniciadas por clientes y cambios de datos de contacto o de credenciales. En móviles el segundo factor puede ser biometría ligada a una clave del almacén seguro del dispositivo (ver SEG-MOV-004).

5.4 SEG-AUT-004 · Protección contra fuerza bruta

Nivel: Obligatorio | **Aplica a:** Web, Móvil, API | **Referencias:** ASVS V2.2.1; OWASP A07:2021; OWASP API4:2023

Tras 5 intentos fallidos consecutivos la cuenta se bloquea 15 minutos. Además, los endpoints de autenticación y de recuperación de contraseña deben tener limitación de tasa por IP y por usuario. Los mensajes de error no deben revelar si el usuario existe.

5.5 SEG-AUT-005 · Cookies de sesión

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** ASVS V3.3.1; ASVS V3.4.1; ASVS V3.4.2; ASVS V3.4.3

Las cookies de sesión o de autenticación deben tener los atributos `Secure`, `HttpOnly` y `SameSite=Strict` (o `Lax` cuando el flujo OIDC lo requiera). La sesión expira tras 20 minutos de inactividad y tiene una duración absoluta máxima de 8 horas. El cierre de sesión invalida la sesión en el servidor. Está prohibido guardar tokens de acceso en `localStorage` o `sessionStorage`.

Ejemplo incorrecto

```
Response.Cookies.Append("sesion", token);   // sin Secure, HttpOnly ni SameSite
```

Ejemplo correcto

```
Response.Cookies.Append("sesion", token, new CookieOptions
{
    Secure = true,
    HttpOnly = true,
    SameSite = SameSiteMode.Strict,
    Expires = DateTimeOffset.UtcNow.AddHours(8)
});
```

5.6 SEG-AUT-006 · Validación de tokens JWT

Nivel: Obligatorio | **Aplica a:** API, Web, Móvil | **Referencias:** OWASP A07:2021; OWASP API2:2023

Toda API que reciba JWT debe validar firma, emisor (`iss`), audiencia (`aud`) y vigencia (`exp`, `nbf`) con una tolerancia de reloj máxima de 2 minutos. Está prohibido aceptar el algoritmo `none` o desactivar cualquiera de las validaciones. El token de acceso dura como máximo 60 minutos; los tokens de refresco se rotan en cada uso.

Ejemplo incorrecto

```
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = false,
    ValidateAudience = false,
    ValidateLifetime = false
};
```

Ejemplo correcto

```
options.Authority = configuracion["Auth:Authority"];
options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true,
    ValidAudience = configuracion["Auth:Audience"],
    ValidateLifetime = true,
    ClockSkew = TimeSpan.FromMinutes(2)
};
```

6. Control de acceso

6.1 SEG-ACC-001 · Autorización en el servidor

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V4.1.1; OWASP A01:2021

Toda decisión de autorización se aplica en el servidor. Ocultar un botón o una ruta en Angular, o deshabilitar un control en una aplicación de escritorio, es solo experiencia de usuario y no constituye un control de acceso.

6.2 SEG-ACC-002 · Verificación de propiedad del recurso

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** ASVS V4.2.1; OWASP A01:2021; OWASP API1:2023

Cuando una operación recibe el identificador de un recurso (cuenta, pedido, documento), el servidor debe verificar que el usuario autenticado tiene derecho sobre ese recurso específico, no solo que tiene el rol adecuado. Esto previene el acceso a datos de otros usuarios cambiando el identificador (IDOR / BOLA).

Ejemplo incorrecto

```
[HttpGet("cuentas/{cuentaId}")]
public async Task<ActionResult<CuentaDto>> Obtener(int cuentaId)
    => await _cuentas.ObtenerAsync(cuentaId);   // cualquier usuario ve cualquier cuenta
```

Ejemplo correcto

```
[HttpGet("cuentas/{cuentaId}")]
public async Task<ActionResult<CuentaDto>> Obtener(int cuentaId)
{
    var cuenta = await _cuentas.ObtenerAsync(cuentaId);
    if (cuenta is null || cuenta.ClienteId != User.ObtenerClienteId())
        return NotFound();                      // 404: no revelar que existe
    return cuenta;
}
```

6.3 SEG-ACC-003 · Protección contra CSRF

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** ASVS V4.2.2; OWASP A01:2021

Las aplicaciones que autentican con cookies deben usar tokens antifalsificación en todas las operaciones que modifican estado (`[ValidateAntiForgeryToken]` o `AutoValidateAntiforgeryToken`). En Angular se usa el soporte XSRF de `HttpClient` con la cookie `XSRF-TOKEN`.

7. Validación de entradas y codificación de salidas

7.1 SEG-INY-001 · Consultas parametrizadas

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V5.3.4; OWASP A03:2021

Toda consulta a base de datos debe usar parámetros. Está prohibido construir sentencias SQL concatenando o interpolando valores provenientes del usuario, tanto en C# como dentro de procedimientos almacenados. El SQL dinámico en T-SQL solo se permite con `sp_executesql` y parámetros; los nombres de columnas u ordenamientos dinámicos se validan contra una lista blanca.

Ejemplo incorrecto

```
var sql = "SELECT ClienteId, Nombre FROM ventas.Cliente WHERE Documento = '" + documento + "'";
using var cmd = new SqlCommand(sql, conexion);

-- T-SQL
SET @sql = N'SELECT * FROM ventas.Cliente WHERE Nombre LIKE ''%' + @filtro + '%''';
EXEC(@sql);
```

Ejemplo correcto

```
const string sql = "SELECT ClienteId, Nombre FROM ventas.Cliente WHERE Documento = @Documento";
using var cmd = new SqlCommand(sql, conexion);
cmd.Parameters.Add("@Documento", SqlDbType.VarChar, 15).Value = documento;

-- T-SQL
SET @sql = N'SELECT ClienteId, Nombre FROM ventas.Cliente WHERE Nombre LIKE @patron';
EXEC sp_executesql @sql, N'@patron NVARCHAR(102)', @patron = N'%' + @filtro + N'%';
```

7.2 SEG-INY-002 · Validación de entradas por lista blanca

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V5.1.3; ASVS V5.1.4; MASVS-CODE-4

Toda entrada externa (formularios, parámetros de URL, cabeceras, archivos, mensajes de colas, respuestas de terceros) se valida en el servidor por tipo, longitud, rango y formato permitido. La validación en el cliente (Angular, WPF) es complementaria y nunca reemplaza a la del servidor.

7.3 SEG-INY-003 · Prevención de XSS

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** ASVS V5.3.3; OWASP A03:2021

Los datos se codifican según el contexto de salida. En Angular está prohibido usar `bypassSecurityTrustHtml`, `bypassSecurityTrustScript` o `bypassSecurityTrustUrl` con datos que provengan del usuario o de una API, y asignar `innerHTML` con contenido no sanitizado. En Razor está prohibido `Html.Raw` con datos no confiables.

Ejemplo incorrecto

```
// comentario.component.ts
this.html = this.sanitizer.bypassSecurityTrustHtml(comentario.texto);
// plantilla: <div [innerHTML]="html"></div>
```

Ejemplo correcto

```
// Se muestra como texto: Angular codifica automáticamente
// plantilla: <p>{{ comentario.texto }}</p>
```

7.4 SEG-INY-004 · Ejecución de comandos del sistema operativo

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V5.3.8; OWASP A03:2021

Está prohibido ejecutar comandos del sistema operativo construidos con datos del usuario (`Process.Start` con una cadena concatenada, `cmd.exe /c`, PowerShell). Si es imprescindible, se invoca un ejecutable fijo con `ProcessStartInfo.ArgumentList` y argumentos validados por lista blanca.

7.5 SEG-INY-005 · Deserialización segura

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V5.5.3; OWASP A08:2021

Está prohibido usar `BinaryFormatter`, `SoapFormatter`, `NetDataContractSerializer` y `LosFormatter`. Con Newtonsoft.Json está prohibido `TypeNameHandling` distinto de `None`. El serializador por defecto es `System.Text.Json`.

Ejemplo incorrecto

```
var formatter = new BinaryFormatter();
var pedido = (Pedido)formatter.Deserialize(stream);
```

Ejemplo correcto

```
var pedido = await JsonSerializer.DeserializeAsync<PedidoDto>(stream, opciones);
```

7.6 SEG-INY-006 · Procesamiento seguro de XML

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V5.5.2; OWASP A05:2021

Los lectores de XML deben tener deshabilitado el procesamiento de DTD y la resolución de entidades externas (`DtdProcessing = DtdProcessing.Prohibit`, `XmlResolver = null`) para prevenir ataques XXE.

7.7 SEG-INY-007 · Carga de archivos

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** ASVS V12.1.1; ASVS V12.3.1; OWASP A04:2021

Los archivos subidos por usuarios tienen un tamaño máximo de 10 MB salvo excepción documentada; se valida la extensión contra una lista blanca y el tipo real por su firma (magic number); el nombre con el que se guarda lo genera el servidor (GUID) y el almacenamiento está fuera de la carpeta pública del sitio (`wwwroot`). Los archivos se analizan con el antivirus corporativo antes de quedar disponibles.

8. Criptografía y gestión de secretos

8.1 SEG-CRI-001 · Algoritmos criptográficos aprobados

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V6.2.2; OWASP A02:2021; MASVS-CRYPTO-1

Algoritmos aprobados: AES-256 en modo GCM para cifrado simétrico; RSA de 3072 bits o más, o ECDSA P-256, para firma y cifrado asimétrico; SHA-256 o superior para hash. Están prohibidos DES, 3DES, RC4, MD5 y SHA-1 para fines de seguridad, y el modo ECB.

8.2 SEG-CRI-002 · Números aleatorios seguros

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V6.3.1; OWASP A02:2021

Tokens, códigos de verificación, identificadores de sesión, sales y claves se generan con `RandomNumberGenerator` (.NET) o `crypto.getRandomValues` (navegador). Está prohibido usar `System.Random` o `Math.random()` para valores de seguridad.

Ejemplo incorrecto

```
var codigo = new Random().Next(100000, 999999).ToString();
```

Ejemplo correcto

```
var codigo = RandomNumberGenerator.GetInt32(100000, 1000000).ToString();
```

8.3 SEG-CRI-003 · Secretos fuera del código

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V6.4.1; OWASP A05:2021

Cadenas de conexión con contraseña, claves de API, certificados y secretos de cliente se guardan en Azure Key Vault y se leen en tiempo de ejecución, preferentemente con identidad administrada. En desarrollo local se usa `dotnet user-secrets`. Está prohibido guardar secretos en el código fuente, en `appsettings.json`, en `environment.ts` de Angular o en cualquier archivo del repositorio. Un secreto expuesto en el repositorio se considera comprometido y debe rotarse.

Ejemplo incorrecto

```
// appsettings.json
"ConnectionStrings": {
  "Ventas": "Server=sqlprod01;Database=Ventas;User Id=app_ventas;Password=Ventas2024!"
}
```

Ejemplo correcto

```
// Program.cs
builder.Configuration.AddAzureKeyVault(
    new Uri(builder.Configuration["KeyVault:Uri"]!), new DefaultAzureCredential());
// El secreto "ConnectionStrings--Ventas" vive en Key Vault
```

8.4 SEG-CRI-004 · Cifrado en tránsito

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V9.1.1; ASVS V14.4.5; OWASP A02:2021; MASVS-NETWORK-1

Toda comunicación usa TLS 1.2 o superior (preferentemente TLS 1.3), incluida la comunicación interna entre servicios y la conexión a SQL Server (`Encrypt=True`). Los sitios web publican HSTS con `max-age` de al menos un año. Está prohibido desactivar la validación de certificados.

8.5 SEG-CRI-005 · Protección de datos sensibles en base de datos

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V8.1; OWASP A02:2021

Las bases de datos de producción usan TDE. Las columnas con datos sensibles (número de documento, número de tarjeta, datos de salud) se protegen además con Always Encrypted o cifrado a nivel de aplicación. Los números de tarjeta completos no se almacenan (se guarda el token del proveedor de pagos). Las copias a ambientes no productivos se enmascaran.

9. Registro de eventos, errores y monitoreo

9.1 SEG-LOG-001 · No registrar datos sensibles

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V7.1.1; OWASP A09:2021; MASVS-STORAGE-2

Está prohibido escribir en logs contraseñas, tokens, cookies de sesión, números de tarjeta, códigos de verificación y el número completo de documento de identidad. Si se necesita trazabilidad, se registra un valor enmascarado (por ejemplo los últimos 4 dígitos).

Ejemplo incorrecto

```
_logger.LogInformation("Login de {Usuario} con clave {Clave}", usuario, clave);
_logger.LogDebug("Token emitido: {Token}", token);
```

Ejemplo correcto

```
_logger.LogInformation("Login exitoso de {Usuario}", usuario);
```

9.2 SEG-LOG-002 · Registro de eventos de seguridad

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V7.1.3; ASVS V7.1.4; OWASP A09:2021

Se registran como mínimo: inicios de sesión exitosos y fallidos, cierres de sesión, accesos denegados, cambios de permisos o roles, cambios de credenciales y errores de validación de entrada repetidos. Cada evento incluye fecha y hora UTC, usuario, IP de origen, aplicación e identificador de correlación, y se envía al SIEM corporativo.

9.3 SEG-LOG-003 · Mensajes de error genéricos

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** ASVS V7.4.1; OWASP A05:2021

Los errores mostrados al usuario o devueltos por una API son genéricos e incluyen un identificador de seguimiento. Está prohibido devolver trazas de pila, sentencias SQL, rutas de archivos o versiones de componentes. En ASP.NET Core la página de excepciones para desarrolladores solo se habilita en el entorno Development.

10. Configuración segura y dependencias

10.1 SEG-CFG-001 · Cabeceras de seguridad HTTP

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** ASVS V14.4.3; ASVS V14.4.4; ASVS V14.4.7; OWASP A05:2021

Los sitios web y las APIs envían: `Content-Security-Policy` (sin `unsafe-inline` ni `unsafe-eval` en scripts), `X-Content-Type-Options: nosniff`, `frame-ancestors 'none'` o `X-Frame-Options: DENY`, `Referrer-Policy: strict-origin-when-cross-origin` y `Strict-Transport-Security`. Se elimina la cabecera `Server` y `X-Powered-By`.

10.2 SEG-CFG-002 · Política CORS

Nivel: Obligatorio | **Aplica a:** API | **Referencias:** ASVS V14.5.3; OWASP A05:2021; OWASP API8:2023

CORS se configura con una lista blanca explícita de orígenes por entorno. Está prohibido `AllowAnyOrigin()` en APIs que usan cookies o credenciales y está prohibido reflejar el encabezado `Origin` recibido.

Ejemplo incorrecto

```
builder.Services.AddCors(o => o.AddDefaultPolicy(p =>
    p.SetIsOriginAllowed(_ => true).AllowCredentials().AllowAnyHeader()));
```

Ejemplo correcto

```
builder.Services.AddCors(o => o.AddDefaultPolicy(p =>
    p.WithOrigins(builder.Configuration.GetSection("Cors:Origenes").Get<string[]>()!)
     .AllowCredentials().WithHeaders("Content-Type", "Authorization")));
```

10.3 SEG-CFG-003 · Gestión de dependencias vulnerables

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** ASVS V14.2.1; OWASP A06:2021; MASVS-CODE-3

El pipeline ejecuta `dotnet list package --vulnerable --include-transitive` y `npm audit` en cada compilación. Una vulnerabilidad crítica o alta bloquea el pase a producción. Está prohibido usar componentes sin soporte del fabricante (por ejemplo versiones de .NET o Angular fuera de soporte).

10.4 SEG-CFG-004 · Análisis de seguridad en el pipeline

Nivel: Obligatorio | **Aplica a:** Web, Móvil, Escritorio, API | **Referencias:** OWASP A04:2021; OWASP A06:2021

Cada pull request ejecuta análisis estático de seguridad (SAST) y detección de secretos. Antes del primer pase a producción y en cada versión mayor se ejecuta un análisis dinámico (DAST) sobre el ambiente de certificación. Los hallazgos críticos y altos deben corregirse antes del pase.

11. Aplicaciones móviles

Las reglas de este capítulo complementan las anteriores para aplicaciones .NET MAUI, Android (Kotlin) e iOS (Swift). Las aplicaciones que realizan transacciones monetarias deben cumplir además el perfil MAS-L2 y los controles de resiliencia (ver EST-REF-001).

11.1 SEG-MOV-001 · Almacenamiento seguro en el dispositivo

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-STORAGE-1; MASVS-CRYPTO-2

Tokens, credenciales y claves se guardan únicamente en el almacén seguro de la plataforma: `SecureStorage` en .NET MAUI, Keychain en iOS y Android Keystore / EncryptedSharedPreferences en Android. Está prohibido guardarlos en `Preferences`, `SharedPreferences`, `UserDefaults`, archivos o bases SQLite sin cifrar.

Ejemplo incorrecto

```
Preferences.Default.Set("access_token", token);
```

Ejemplo correcto

```
await SecureStorage.Default.SetAsync("access_token", token);
```

11.2 SEG-MOV-002 · Prevención de fugas de datos

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-STORAGE-2; MASVS-PLATFORM-3

En pantallas con datos sensibles se impiden las capturas de pantalla y la vista previa en el selector de aplicaciones (`FLAG_SECURE` en Android, vista de privacidad en iOS). Los datos sensibles no se escriben en logs del dispositivo, no se copian al portapapeles sin acción del usuario y los archivos de la aplicación se excluyen de las copias de seguridad automáticas.

11.3 SEG-MOV-003 · Comunicación segura y certificate pinning

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-NETWORK-1; MASVS-NETWORK-2

Las aplicaciones móviles usan solo HTTPS y aplican certificate pinning (clave pública) para las APIs de la empresa, con al menos un pin de respaldo. Está prohibido aceptar cualquier certificado en el validador.

Ejemplo incorrecto

```
var handler = new HttpClientHandler
{
    ServerCertificateCustomValidationCallback = (_, _, _, _) => true
};
```

Ejemplo correcto

```
var handler = new HttpClientHandler
{
    ServerCertificateCustomValidationCallback = (_, cert, _, errores) =>
        errores == SslPolicyErrors.None && PinsPermitidos.Contains(cert!.GetPublicKeyString())
};
```

11.4 SEG-MOV-004 · Autenticación biométrica

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-AUTH-2

La biometría local debe estar ligada a una operación criptográfica (una clave en Keystore/Keychain que exige autenticación biométrica para usarse). Está prohibido implementar la biometría como un simple valor verdadero/falso que la aplicación consulta.

11.5 SEG-MOV-005 · WebViews

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-PLATFORM-2

Los WebView solo cargan contenido de dominios de la empresa por HTTPS, con JavaScript deshabilitado salvo necesidad documentada y sin puentes JavaScript-nativo expuestos a contenido no confiable.

11.6 SEG-MOV-006 · Resiliencia ante ingeniería inversa

Nivel: Recomendado | **Aplica a:** Móvil | **Referencias:** MASVS-RESILIENCE-1; MASVS-RESILIENCE-2; MASVS-RESILIENCE-4

Las aplicaciones transaccionales detectan dispositivos con root o jailbreak, depuradores e instrumentación, verifican su propia integridad y ofuscan el código. Ante una detección se informa al backend y se limitan las operaciones de riesgo.

11.7 SEG-MOV-007 · Versiones soportadas y actualización forzada

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-CODE-1; MASVS-CODE-2

Se define una versión mínima de sistema operativo con soporte de seguridad del fabricante. El backend puede exigir una versión mínima de la aplicación y bloquear versiones con vulnerabilidades conocidas.

11.8 SEG-MOV-008 · Minimización de permisos y privacidad

Nivel: Obligatorio | **Aplica a:** Móvil | **Referencias:** MASVS-PRIVACY-1; MASVS-PRIVACY-3

La aplicación solicita solo los permisos del sistema que necesita, en el momento en que los necesita, y explica al usuario para qué. No se recolectan identificadores del dispositivo para fines distintos a la seguridad.

12. Aplicaciones de escritorio

12.1 SEG-DSK-001 · Acceso a datos mediante servicios

Nivel: Obligatorio | **Aplica a:** Escritorio | **Referencias:** OWASP A01:2021; OWASP A04:2021

Las aplicaciones de escritorio nuevas acceden a los datos a través de una API corporativa y no directamente a SQL Server. Se permite la conexión directa solo en aplicaciones de uso interno en la red corporativa, con autenticación integrada de Windows y un usuario de base de datos con permisos limitados a procedimientos almacenados (ver EST-DSK-001).

12.2 SEG-DSK-002 · Protección de datos locales

Nivel: Obligatorio | **Aplica a:** Escritorio | **Referencias:** ASVS V6.4.1; OWASP A02:2021

Credenciales, tokens o datos sensibles que la aplicación necesite guardar en el equipo se protegen con DPAPI (`ProtectedData.Protect` con alcance `CurrentUser`). Está prohibido guardar contraseñas o cadenas de conexión con contraseña en texto plano en `app.config`, `appsettings.json`, el registro de Windows o archivos locales.

Ejemplo incorrecto

```
<!-- app.config -->
<add name="Ventas"
     connectionString="Server=sql01;Database=Ventas;User Id=ventas;Password=Clave123" />
```

Ejemplo correcto

```
var cifrado = ProtectedData.Protect(Encoding.UTF8.GetBytes(token), entropia,
                                    DataProtectionScope.CurrentUser);
File.WriteAllBytes(rutaCache, cifrado);
```

12.3 SEG-DSK-003 · Firma de código

Nivel: Obligatorio | **Aplica a:** Escritorio | **Referencias:** OWASP A08:2021

Ejecutables, bibliotecas e instaladores se firman con el certificado Authenticode corporativo. El mecanismo de actualización automática verifica la firma antes de instalar una nueva versión.

12.4 SEG-DSK-004 · Ejecución sin privilegios

Nivel: Obligatorio | **Aplica a:** Escritorio | **Referencias:** ASVS V4.1.3; OWASP A04:2021

Las aplicaciones se ejecutan con los permisos del usuario estándar. Está prohibido requerir privilegios de administrador para el uso normal y escribir en `Program Files` durante la ejecución; los datos de usuario se guardan en `%LOCALAPPDATA%`.

13. Excepciones

Las excepciones a reglas de este estándar se solicitan con el formulario FOR-ARQ-07, incluyendo el riesgo aceptado, los controles compensatorios y la fecha de vencimiento. La vigencia máxima es de 6 meses y no son renovables más de una vez.

14. Verificación del cumplimiento

El cumplimiento se verifica en la revisión de código, en el pipeline (SAST, análisis de dependencias, detección de secretos) y en las pruebas de penetración anuales. Los hallazgos se gestionan según su severidad: crítica 7 días, alta 30 días, media 90 días, baja 180 días.